

ARCHITECTURE DOCUMENT

Project: DeFi

Group: SE - 2438

Team Members: Farkhad Imanbayev, Mendeke Seitbayev, Timur Bizinskiy

TABLE OF CONTENTS

1. Executive Summary
2. System Context Diagram (C4 Level 1)
3. Container / Component Diagram (C4 Level 2)
4. Sequence Diagrams
5. Data Model and Storage Layout
6. Trust Assumptions
7. Design Decisions Log (ADRs)

1. EXECUTIVE SUMMARY

Our project is a protocol with five smart contracts:

- An AMM (constant-product $x \cdot y = k$) for token swaps.
- An ERC-4626 yield vault for protocol fees.
- A lending market with a 75% LTV.
- An ERC20Votes token (DGT) for voting.
- A full OpenZeppelin Governor + Timelock.

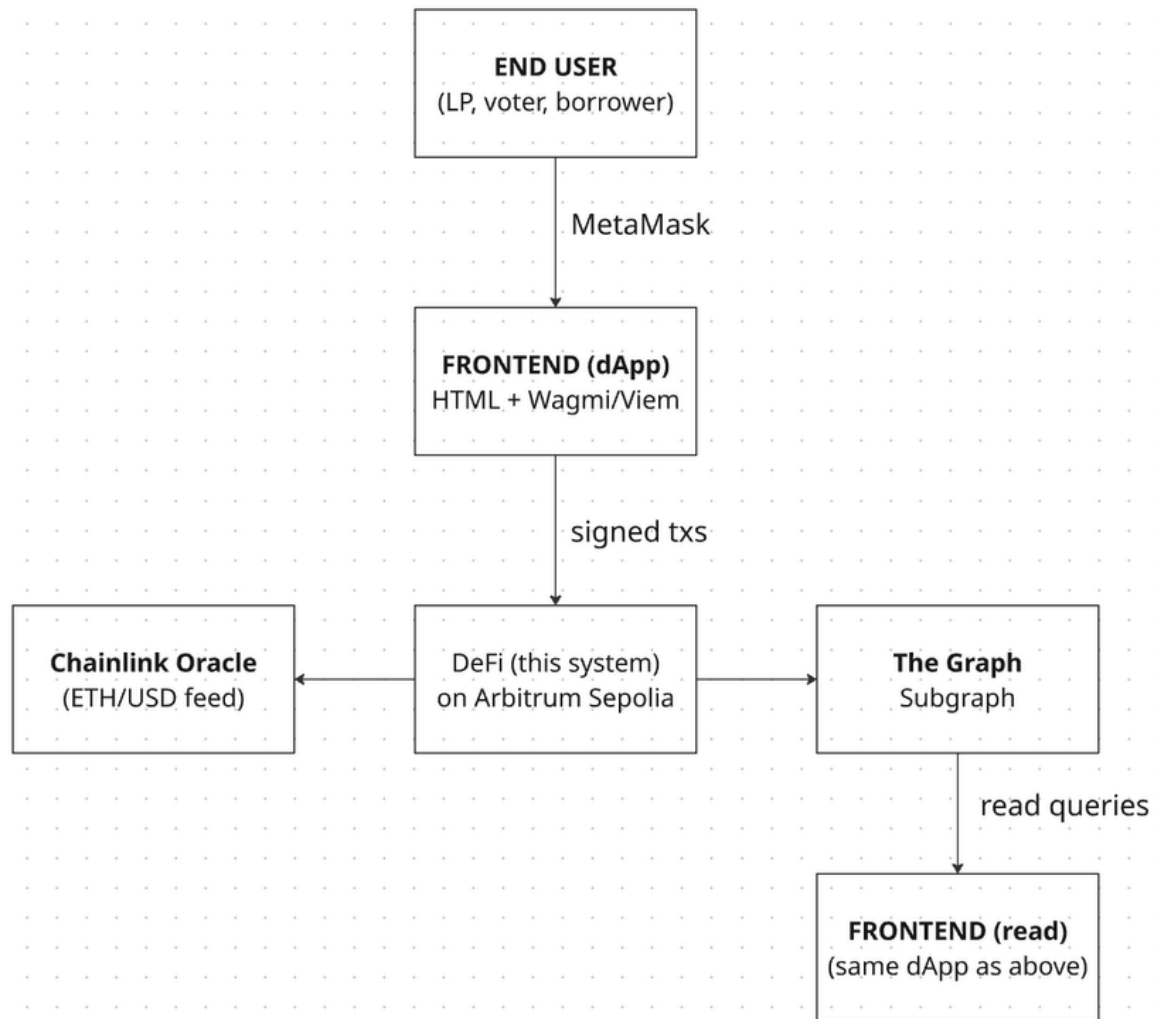
The protocol is owned by the Timelock. After deployment the deployer drops its admin role. The DAO is then autonomous.

It is deployed on Arbitrum Sepolia (chainId 421614). It is indexed by The Graph. The frontend is a Wagmi/Viem dApp.

This document follows the C4 model. It also lists storage layout, trust assumptions, and ADRs.

2. SYSTEM CONTEXT DIAGRAM (C4 LEVEL 1)

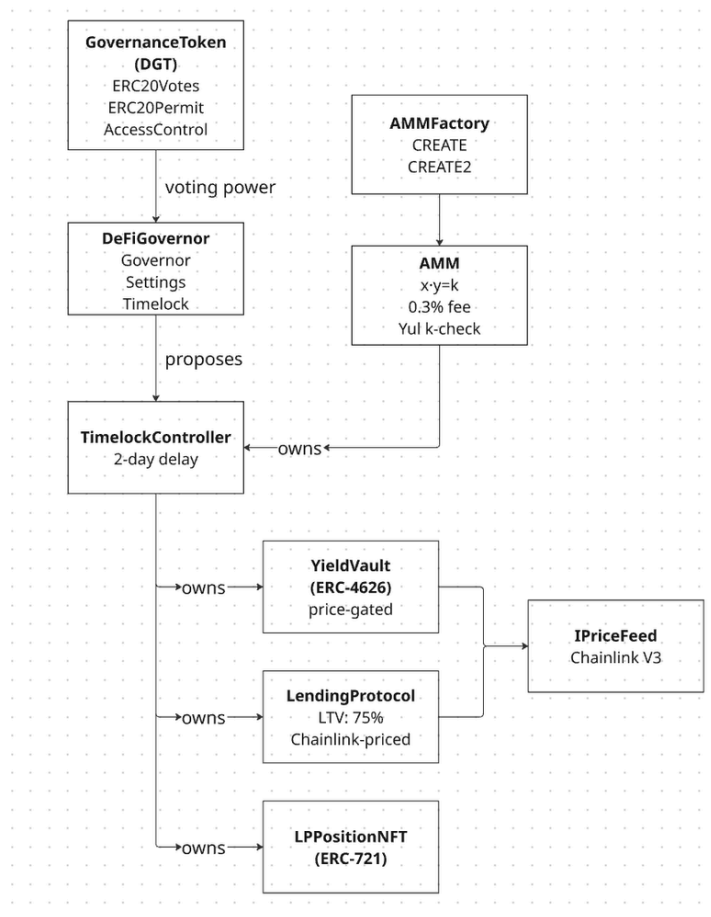
C4 Level 1 shows the system and its environment.



- End user - uses the frontend. Never calls contracts directly.
- Chainlink Oracle - external system trusted for price data. We check the staleness.
- The Graph - external indexer. Used by the frontend for read-only data.
- Arbitrum Sepolia - the L2 testnet we deploy on.

3. CONTAINER / COMPONENT DIAGRAM (C4 LEVEL 2)

C4 Level 2 shows the contracts inside the system.



Roles after deployment:

- TimelockController is the owner of AMM, LendingProtocol, AMMFactory. It also holds DEFAULT_ADMIN_ROLE on GovernanceToken and YieldVault.
- DeFiGovernor has PROPOSER_ROLE on the Timelock.
- The deployer has nothing - it revokes its role at the end of Deploy.s.sol.

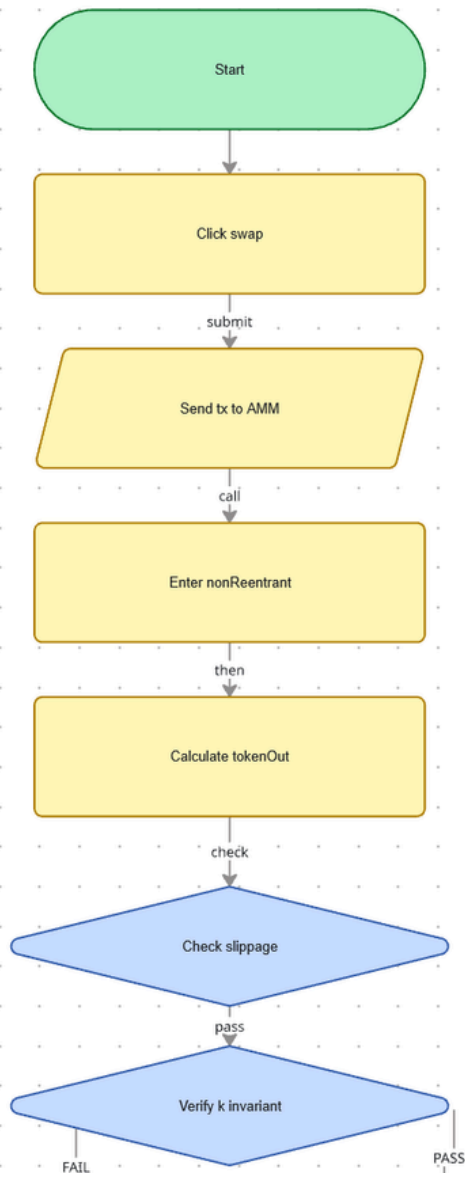
4. SEQUENCE DIAGRAMS

Three flows are documented:

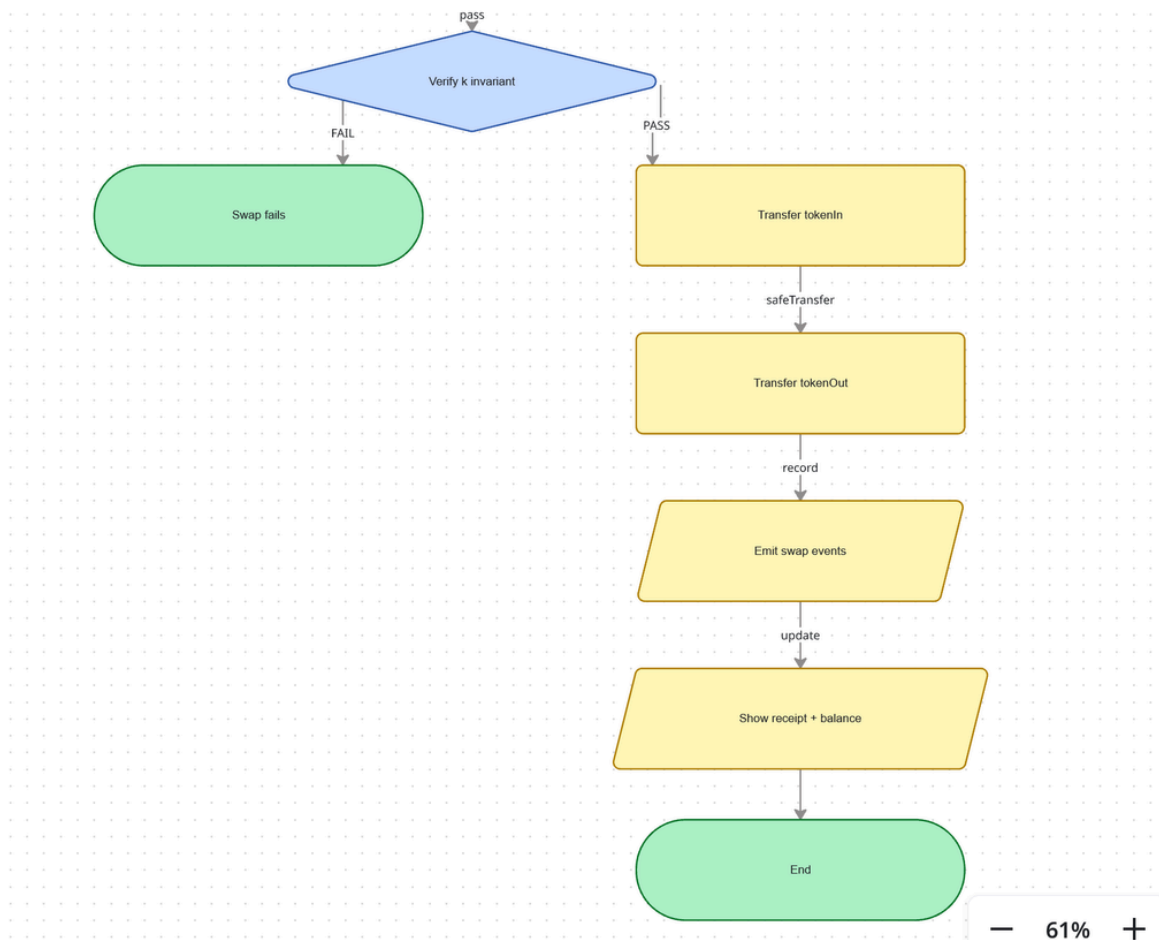
1. Swap on the AMM
2. Propose -> Vote -> Queue -> Execute
3. Deposit -> Borrow

4.1 SWAP ON THE AMM

part 1



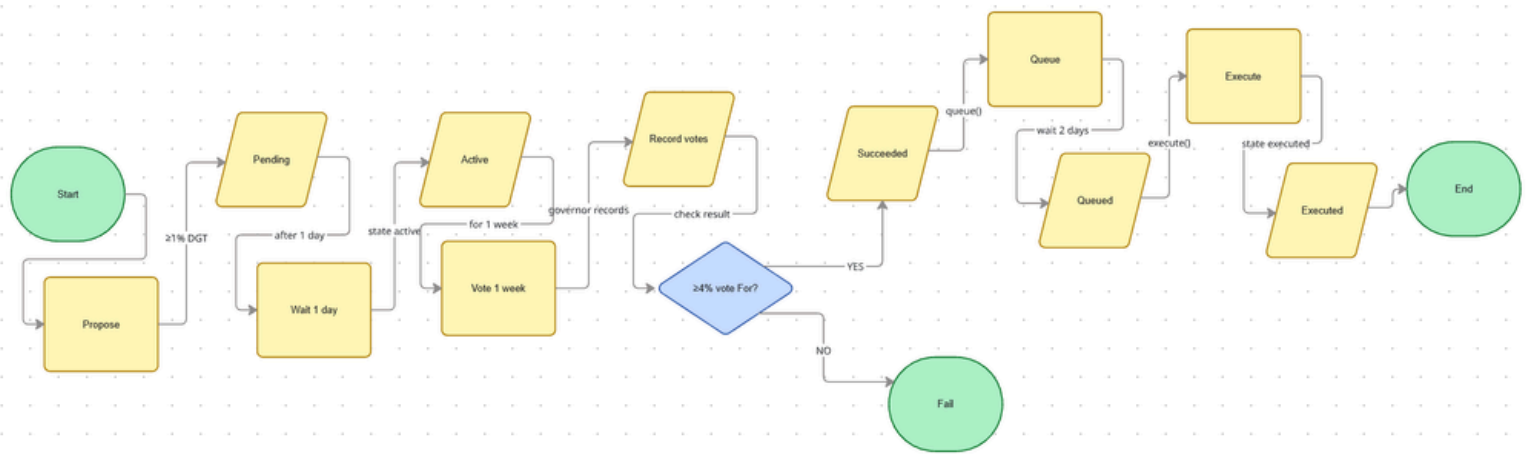
part 2



Sequence:

1. User clicks swap in the website.
2. Wallet sends a swap transaction to the AMM contract.
3. AMM enters nonReentrant – this stops bad tokens from attacking.
4. AMM calculates how much tokenOut the user will get.
5. AMM checks slippage - the user gets at least the minimum they agreed to.
6. AMM uses Yul (a special code) to check that the math is correct (k invariant). If not, the swap fails.
7. AMM takes tokenIn from the user with safeTransferFrom.
8. AMM gives tokenOut to the user with safeTransfer.
9. AMM records the swap by emitting Swap and ReservesUpdated events.
10. Wallet shows the receipt to the user. The user sees their new balance.

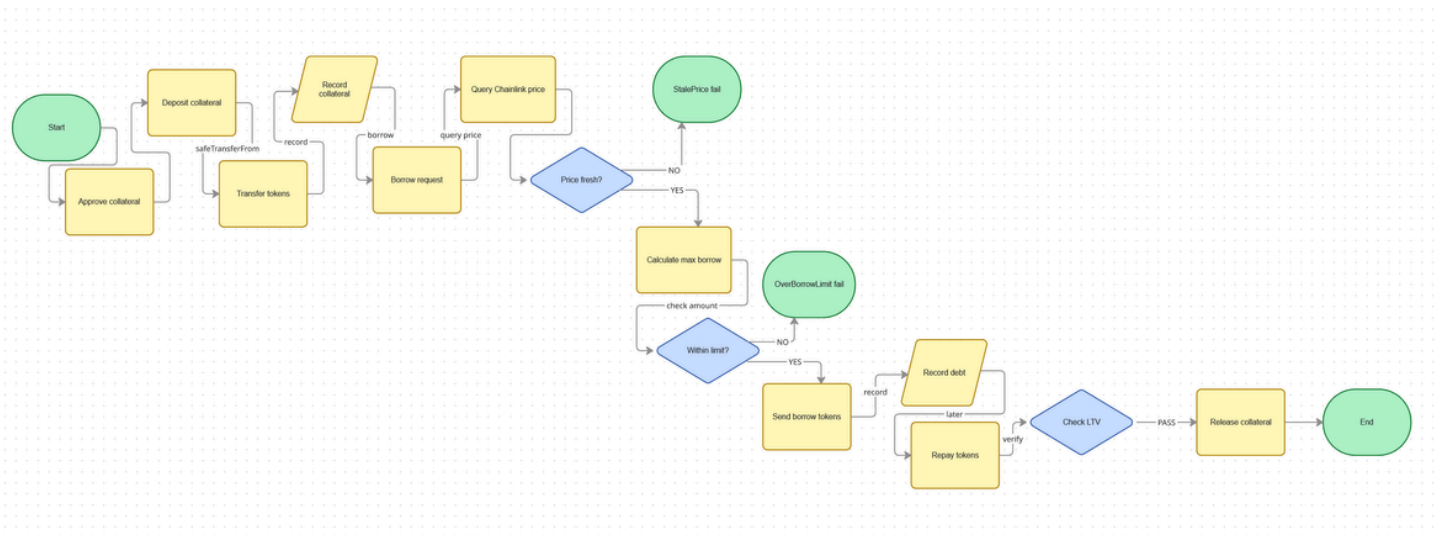
4.2 PROPOSE -> VOTE -> QUEUE -> EXECUTE



Notes:

- Voting delay = 1 day (7 200 blocks).
- Voting period = 1 week (50 400 blocks).
- Quorum = 4% of total supply at snapshot.
- Proposal threshold = 1% - checked with `getPastTotalSupply(clock() - 1) / 100`.
- Timelock min delay = 2 days. Last line of defense.

4.3 DEPOSIT -> BORROW



Notes:

- `getLatestPrice()` reverts with `StalePrice()` if data is older than 1 hour.
- LTV is fixed at 75%. Over-LTV borrow reverts with `OverBorrowLimit()`.
- Collateral withdraw runs the same LTV check on the new balance.

5. DATA MODEL AND STORAGE LAYOUT

This section lists storage for each contract. This is needed to prove that storage slots will not collide between versions of an upgradeable contract.

5.1 AMM.sol

Inherits: ERC20, Ownable, ReentrancyGuard, Pausable.

Slot	Variable	Type	Notes
0	balances	mapping	ERC20
1	allowances	mapping	ERC20
2	totalSupply	uint256	ERC20
3	_name	string	ERC20
4	symbol	string	ERC20
5	owner	address	Ownable
6	_status	uint256	ReentrancyGuard
7	_paused	bool	Pausable
8	tokenA	IERC20	swap token A
9	tokenB	IERC20	swap token B
10	reserveA	uint256	reserve of A
11	reserveB	uint256	reserve of B
12	version	uint256	for V1 → V2

5.2 YieldVault.sol

Inherits: ERC4626, AccessControl, ReentrancyGuard, Pausable.

Slot	Variable	Type	Notes
0-4	(ERC20 fields)	-	balances, allowances, supply, name, symbol
5	_asset	IERC20	ERC4626
6	_underlyingDecimals	uint8	ERC4626
7	_roles	mapping	AccessControl
8	_status	uint256	ReentrancyGuard
9	_paused	bool	Pausable
10	priceFeed	IPriceFeed	Chainlink
11	maxStaleness	uint256	seconds
12	minPrice	uint256	minimum price (8 dec)
13	bypassPriceCheck	bool	testnet flag
14	accruedYield	uint256	extra yield from fees

5.3 LendingProtocol.sol

Slot	Variable	Type	Notes
0	_owner	address	Ownable
1	_status	uint256	ReentrancyGuard
2	positions	mapping	per-user collateral / debt

5.4 GovernanceToken.sol

Inherits: ERC20, ERC20Permit, ERC20Votes, AccessControl.

Slot	Variable	Type	Notes
0-4	(ERC20 fields)	-	ERC20 base
5	_nonces	mapping	from Permit
6	_delegatee	mapping	ERC20Votes
7	_delegateCheckpoints	mapping	ERC20Votes
8	_totalSupplyCheckpoints	array	ERC20Votes
9	_roles	mapping	AccessControl

5.5 DeFiGovernor.sol

All storage is inherited from OpenZeppelin's Governor stack. We add no new state. We only override proposalThreshold() to return 1% of past total supply.

5.6 Storage-collision safety

For upgradeable paths (AMMFactory uses ERC1967Proxy), we follow the OpenZeppelin rule: never reorder or remove fields between versions. Only append. The current state is V1.

6. TRUST ASSUMPTIONS

This section answers: who can do what, and what happens if they are compromised.

6.1 Roles

Role / actor	Powers	Holder after deployment
Deployer EOA	None (revokes itself at end of deploy)	—
TimelockController	DEFAULT_ADMIN_ROLE on token + vault / Ownable of AMM / LendingProtocol / Factory	Controlled by Governor
DeFiGovernor	PROPOSER_ROLE on Timelock (schedule + cancel)	Controlled by token holders (4 percent quorum)
Token holders	Vote on proposals / propose if 1 percent supply	Public
MINTER_ROLE on DGT	Mint up to MAX_SUPPLY (100M)	Timelock
YIELD_MANAGER_ROLE on Vault	Call distributeYield()	Initially admin
PAUSER_ROLE on Vault	Pause or unpause deposits	Timelock

6.2 What can each role do?

The DAO (a successful proposal) can: mint new DGT (up to MAX_SUPPLY), pause/unpause the vault, change vault's price feed, set staleness threshold, pause AMM, upgrade contracts (if the

path exists), drain treasury.

A single token holder can only vote with their delegated power. They cannot block proposals alone.

Anyone can call `execute()` on a queued proposal after the 2-day delay. This is on purpose: a successful proposal cannot be silently dropped.

6.3 What if X is compromised?

Compromise	Impact	Mitigation
A single voter wallet	Their vote weight is lost	Quorum + threshold mean one voter can't push anything alone
A whale gets 4 percent or more + majority	They can pass proposals	2-day Timelock gives users time to exit. Flash-loan governance is blocked by snapshot voting
DeFiGovernor is buggy	Can only schedule on Timelock	Timelock is the last line of defense
TimelockController is buggy	Worst case: everything	We use the unmodified OpenZeppelin version
Deployer EOA still has admin	Could drain treasury	Script revokes the role. <code>VerifyDeployment.s.sol</code> checks this.
Chainlink feed returns stale data	Vault and lending halt	Staleness threshold = 1 hour
Chainlink feed returns bad price	Lending mis-prices LTV positions	Single-feed risk. See ADR-005.

6.4 Centralization disclosure

The protocol is not fully decentralized at launch:

- The vault has `bypassPriceCheck = true` to work around the testnet feed. On mainnet this must be flipped to false.
- LendingProtocol owner is not transferred to the Timelock in the current `Deploy.s.sol` (tracked in audit as I-04).

7. DESIGN DECISIONS LOG (ADRS)

Short records of why we made each major Decision:

ADR-001: Foundry over Hardhat

Context: The spec allows both Foundry and Hardhat. Our test load is heavy in fuzz and invariant tests.

Decision: Use Foundry.

Consequences: Faster test runs. Native Solidity test code. Fuzz and invariant tools out of the box. Trade-off: less mature deployment plugins - we write `Deploy.s.sol` ourselves.

ADR-002: AMM upgradeability

Context: The spec requires at least one UUPS-upgradeable contract.

Decision: Make AMM upgradeable (UUPS, ERC1967 proxy via the AMMFactory).

Consequences: V2 can fix bugs without migration. Storage becomes append-only forever. Upgrade is gated by the Timelock.

Known issue: The current `AMM.sol` uses a constructor, while `AMMFactory.sol` calls `AMM.initialize(...)`. This mismatch is tracked in the audit (H-01).

ADR-003: Yul only for the k-invariant check

Context: The spec requires inline Yul benchmarked against pure Solidity.

Decision: Use Yul only in `AMM._verifyKInvariant`.

Consequences: About 140 gas saved per swap. Audit surface is minimal - only one tiny function uses Yul. Benchmark is in `gas-optimization.md`.

ADR-004: Custom errors over require(string)

Decision: Use custom errors everywhere.

Consequences: About 50 gas saved per revert. Better tooling support (Foundry shows error name in traces).

ADR-005: Single Chainlink feed

Context: A production lending market should aggregate multiple oracles.

Decision: Use a single Chainlink feed with a 1-hour staleness check for V1.

Consequences: Simpler code, easier audit. Known risk: if the feed depegs, the protocol misprices collateral. V2 could add a TWAP fallback.

ADR-006: ERC-721 LP Position is optional

Decision: Implement `LPPositionNFT` as a standalone contract. Not minted by the AMM.

Consequences: Counts toward the ERC-721 requirement. Does not complicate the swap path. Can be wired in later.

ADR-007: Pull-over-push for yield

Decision: AMM does not push fees to the vault. The vault has `distributeYield(amount)`, called after tokens are already transferred in.

Consequences: No external call inside AMM swap. Removes a re-entry vector. Matches "pull-over-push payments" pattern.

ADR-008: Block-based voting periods

Context: On Arbitrum, block times are about 0.25 seconds, not 12 seconds.

Decision: Use 7200 blocks anyway (L1-style).

Consequences: Voting delay on Arbitrum is shorter than 1 day in practice. For mainnet, switch to timestamp-based voting via OZ's clock() override.